Candidate's name: Kayode Oladipo

REFLECTIVE SUMMARY EXPLAINING HOW COPILOT ASSISTED IN EACH STEP

Activity 1: Using Microsoft Copilot to Generate and Refine Integration Code

Copilot helped me to generate code snippets for the API call logic within the blazor FetchProducts.razor on the client side. It also helped me debug and refine the code. It also helped me to refactor and comment the codes. Within the development environment It predicts and suggests what I need for API calls. It also helped me in the writing program.cs for example the relative paths api/products failed to load data when the api was called in the browser so Microsoft AI helped me by suggesting that I should use the full path(to include https) that is https://localhost:5137/

It also enhances integration by offering smart suggestions for managing front-end and back-end communication. For example, when I wanted to retrieve data through an API call co-pilot suggests proposed streamlined workflows to ensure the data is efficiently accessed, properly formatted, and well-organized.

Activity 2: Debugging and Fixing Integration Issues with Copilot

Copilot suggests code snippets to handle some errors for instance when asynchronously calling API it suggests using try and catch to gracefully handle any error that may arise by displaying user friendly error messages if the retrieval doesn't go through e.g by displaying "Error fetching products". This try and catch was used in FetchProducts.razor on the client side app.

Activity 3: Creating and Managing JSON with Microsoft Copilot

Microsoft co-pilot helped to suggest the codes to use to serialise the data especially the product data in JSON format when making API call in the browser in the server side. It also enhances integration by offering smart suggestions for managing front-end and back-end communication. It help me in fetching product information and managing inventory by categorizing the records under product data. This logical grouping simplifies finding and modifying specific workflows, ensuring that changes in one area don't inadvertently affect other areas. Copilot assists with clear naming practices. It does this by analysing the context of your code and generating names that align with common naming conventions and best practices.

Activity 4: Optimizing Integration Code for Performance Using Microsoft Copilot

Microsoft co-pilot suggests improvement that I should make to enhance performance for instance Microsoft co-pilot suggested

add formatting for price that I should declare it as decimal instead of double. This modification was made in the product model in the FetchProducts.razor on the client side. It also suggests that I should consider using a table instead of <ul> for structured data. It also suggests using cache with expiration to enhance performance. It helped me to refactor and comment the programs as well.

Microsoft co-pilot suggests that in-memory caching should be enabled as shown below.

```
builder.Services.AddMemoryCache();
```

Caching reduce redundant API calls in the front-end and the backend as well. Implementing caching strategies in the back-end minimizes server load. By keeping a local copy of the responses, the inventory hub application can quickly retrieve the data without making another call to the server. This approach minimizes redundant requests, improves performance, and reduces server load while providing fasterand more responsive experiences for the users. To implement caching, some of the products in the product- list in the server side were identified as data that would be frequently accessed.

Microsoft co-pilot suggests how the middleware CORS (cross origin resource sharing) should be used. For instance it advised that the following code snippet is risky should not be used:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});
```

Microsoft co-pilot suggests that the following code snippet should be used instead

```
.AllowAnyOrigin()
.WithOrigins("https://localhost:5137")
```

That .WithOrigins("https://localhost:5137") is permissive contrary wise .AllowAnyOrigin() is risky to use within a production environment.

Microsoft co-pilot helped me to generate a code snippet for creating nested JSON data format. This was used in the application.

Refactor any repetitive or inefficient code in both projects.

The changes have occurred are :

- Added comments where they matter
- Service registration
- Endpoint behaviour
- Cache logic
- Safer, reusable, matches Blazor Product model in the Blazor FetchProduct.cs
- Used decimal for Price, Correct for financial values
- Explained cache strategy inline
- Makes maintenance much easier later

Overall Microsoft co-pilot makes programming a fun, and it helps to make application development very rapid, precise and efficient. It saves time makes application development less laborious. Microsoft assisted me to receive and apply feedback for errors. When Co-pilot detected an error, I reviewed its feedback and suggested fixes for syntax or logic issues directly within development environment.

Request debugging assistance: I Use Co-pilot to get detailed explanations of errors and ask for step-by-step guidance to troubleshoot complex bugs.

Microsoft co-pilot enables real-time monitoring of my codes because I have embedded Microsoft co-pilot within my visual studio code integrated development: While writing my codes, It monitors it in real time. It will automatically flag potential bugs, logical errors, or problematic patterns as I code suggesting possible code snippets that may be relevant to the problem I am solving.